

Building Classifiers with Independency Constraints

Toon Calders

Faisal Kamiran

Mykola Pechenizkiy

Eindhoven University of Technology, The Netherlands

E-mail: {t.calders, f.kamiran, m.pechenizkiy}@tue.nl

Abstract

In classifier learning, the quality of the final model depends critically on the quality of the training data. In this paper we study the case where the input data contains unjustified dependencies between some data attributes and the class labels. Such cases arise for example when the training data is collected from different sources with different criteria for labeling or when the data is generated by a biased decision process. When a classifier is trained directly on such data, these undesirable dependencies will carry over to the classifier's predictions. In order to tackle this problem, we study the classification with independence constraints problem: find an accurate model for which the predictions are independent from a given binary attribute. In this way we obtain a classifier that is as faithful to the data as possible but does not have the undesired bias. We propose two solutions for this problem and present a thorough empirical validation.

1. Introduction

Motivation. Classifier construction is one of the most researched topics within the data mining and machine learning communities. Literally thousands of algorithms have been proposed. The quality of the learned models, however, depends critically on the quality of the training data. No matter which classifier inducer is applied, if the training data is incorrect, poor models will result. In this paper we want to study cases in which the input data contains dependencies between some of its attributes and the class labels that are either incorrect, or undesirable. Such cases naturally occur when, e.g., the decision process leading to the labels was biased, e.g., due to discrimination as illustrated by the next example. *Throughout the years, an employment bureau recorded various parameters of job candidates. Based on these parameters, the company wants to learn a model for partially automating the match-making between a job and a job candidate. A match is labeled as successful if the company hires the applicant. It turns out, however, that*

the historical data is biased; for higher board functions, Caucasian males are systematically being favored. A model learned directly on this data will learn this discriminatory behavior and apply it over future predictions. From an ethical and legal point of view it is of course unacceptable that a model discriminating in this way is deployed; instead, it is preferable that the class assigned by the model is independent of sex and ethnicity. In this case it is desirable to have a mean to “tell” the algorithm that its predictions should be independent with the attributes sex and ethnicity.

As illustrated by the next example, the use of independence constraints is not restricted to discrimination-aware datamining. A survey is being conducted by a team of researchers; each researcher visits a number of regionally co-located hospitals and enquires some patients. The survey contains ambiguous questions (e.g., “Is the patient anxious?”, “Is the patient suffering from delusions?”). Different enquirers will answer to these questions in different ways. Generalizing directly from the training set consisting of all surveys without taking into account these differences among the enquirers may easily result in misleading findings. For example, if many surveys from hospitals in a particular area A are supplied by an enquirer who more quickly than the others diagnoses anxiety symptoms, faulty conclusions such as “Patients in area A suffer from anxiety symptoms more often than other patients” may emerge. Whereas in the discrimination example the data was correct (the label reflects whether someone did or did not get the job) and the independence constraints were useful from a utility point of view, in the hospital survey example the independence constraints are a useful tool to avoid over-fitting the classifier by requiring that the learned classifier does not correlate with the enquirer.

Independence constraint. The original idea of requiring independent predictions stems from [5] where it was introduced in the context of discrimination. Here we will concentrate on the case where a labeled dataset is given, and one binary attribute B which we do not want the predictions to correlate with. The dependence of the predictions

of a classifier C on the attribute B is defined as

$$P(C(x) = + | x(B) = 1) - P(C(x) = + | x(B) = 0)$$

for an unseen tuple x . A positive dependence will reflect that a tuple for which B is 1 has a higher chance of being assigned the positive label by C than one where B equals 0. The problem of classification with independency constraints is not a trivial one. The straightforward solution of removing the attribute B from the training-set does in most cases not solve this problem at all. For example, removing *sex* and *ethnicity* for the job-matching example might not help, as other attributes may be correlated with it. For example, residential area can be highly correlated with ethnicity. Blindly applying an out-of-the-box classifier on the job-matching data without the ethnicity attribute will in such a situation still lead to a model that discriminates indirectly based on residential area. A parallel can be drawn with the practise of *redlining*: denying inhabitants of certain racially determined areas from services such as loans.

Solutions. The problem of classification with independency constraints is in fact a multi-objective optimization problem; on the one hand the more dependence we allow for, the higher accuracy we can obtain and on the other hand, in general we can trade in accuracy in order to reduce the dependence. In the paper we start with a theoretical study of this trade-off. Then the following two methods for incorporating independency constraints into the classifier construction process will be discussed:

1. **Massaging the dataset.** We start by changing some labels in the dataset in order to remove the dependency between the labels and the attribute B . For the relabeling a ranker is used. On the cleaned dataset the final classifier is learned. This method is an extension of the method proposed in [5] where only a Naive Bayesian classifier was considered for both the ranking and learning. In this paper we consider arbitrary combinations of ranker and learner.
2. **Reweighting.** Instead of changing the labels, the tuples in the training dataset are assigned weights. As we will show, by carefully choosing the weights the training dataset can be balanced w.r.t. B without having to change any of the labels. Many classification methods are based on frequencies. In these methods the weights can be used directly in the frequency counts leading to a more independent classification.

Experiments. A thorough empirical study was conducted. In the paper we present the results of experiments supporting the following claims:

- (i) The redlining effect is indeed present; removing the attribute B from the dataset does not always result in the removal of the undesired dependency.
- (ii) Especially the *Massaging* leads to an effective decrease in dependence with a minimal decrease in accuracy.

Outline. The rest of the paper is organized as follows. In Section 2 we formally define the problem statement and make a theoretical analysis of the trade-off between accuracy and dependence. In Section 3, the two different approaches towards the problem are discussed. These solutions are empirically evaluated in Section 4 and Section 5 describes related work. Section 6 concludes the paper and gives directions for further work.

2. Problem Statement

In this section we formally introduce the notion of an independency constraint and we theoretically analyze the trade-off between accuracy and dependence.

2.1. Independency Constraints

We assume a set of attributes $\{A_1, \dots, A_n\}$ and their respective domains $dom(A_i)$, $i = 1 \dots n$ have been given. A *tuple* over the schema $(A_1, \dots, A_n)$ is an element of $dom(A_1) \times \dots \times dom(A_n)$. A dataset over the schema $(A_1, \dots, A_n)$ is a finite set of such tuples and a labeled dataset is a finite set of tuples over the schema $(A_1, \dots, A_n, Class)$. Throughout the paper we will assume $dom(Class) = \{-, +\}$.

Let a labeled database D , an attribute B and a value $b \in dom(B)$ be given. The *dependence* between $B = b$ and $Class$ in D , $dep_{B=b}(D)$, is defined as the difference of the probability of being in the positive class between the tuples having $B = b$ in D and those having $B \neq b$ in D ; that is:

$$dep_{B=b}(D) := \frac{|\{x \in D \mid x(B) \neq b, x(Class) = +\}|}{|\{x \in D \mid x(B) \neq b\}|} - \frac{|\{x \in D \mid x(B) = b, x(Class) = +\}|}{|\{x \in D \mid x(B) = b\}|}.$$

(When clear from the context we will omit $B = b$ from the subscript.) A positive dependence means that tuples with $B = b$ are less likely to be in the positive class than tuples with $B \neq b$.

In Table 1, an example dataset is given. This dataset contains the *Sex*, *Ethnicity*, *Highest Degree* of 10 job applicants, the *Job Type* they applied for and the *Class* defining the outcome of the selection procedure. In this

Sex	Ethnicity	Highest Degree	Job Type	Class
m	native	h. school	board	+
m	native	univ.	board	+
m	native	h. school	board	+
m	non-nat.	h. school	healthcare	+
m	non-nat.	univ.	healthcare	-
f	non-nat.	univ.	education	-
f	native	h. school	education	-
f	native	none	healthcare	+
f	non-nat.	univ.	education	-
f	native	h. school	board	+

Table 1. Sample relation for the job-application example.

dataset, the dependence ratio between Sex and $Class$ will be $dep_{Sex=f}(D) := \frac{4}{5} - \frac{2}{5} = 40\%$. It means that the data object with $Sex = f$ will have 40% less chance of getting a job than the one with $Sex = m$.

2.2. Classifying with Independency Constraints

The problem we study in the paper is now as follows: given a labeled dataset D an attribute B , and a value $b \in dom(B)$, learn a classifier C such that:

- (a) the accuracy of C for future predictions is high; and
- (b) the dependence between $B = b$ and $Class$ is low.

Clearly there will be a trade-off between the accuracy and the dependence of the classifier. In general, lowering the dependence will result in lowering the accuracy as well and vice versa. This trade-off is further elaborated upon in the next subsection. In this paper we are making three strong assumptions:

- A1 We are implicitly assuming that the primary intention is learning the most accurate classifier for which the dependence is 0.
- A2 Ideally the learned classifier should not use the attribute B to make its predictions.
- A3 The total ratio of positive predictions of the learned classifier should be similar to the ratio of positive labels in the dataset D .

We do not claim that other settings where these assumptions are violated are not of interest, but at the current stage our work is restricted to this setting.

2.3. Accuracy - Independence Trade-Off

Before going into the proposed solutions, we first theoretically study the trade-off between independence and accuracy in a general setting. We will call a classifier *optimal w.r.t. dependence and accuracy (DA-optimal)* in a set of classifiers $\mathcal{C}$ if it is an element of that class and there does not exist another classifier in that class $\mathcal{C}$ with at the same time a lower dependence and a higher accuracy.

Perfect Classifiers. We will use $\mathcal{C}_{all}$ to denote the set of all classifiers and $\mathcal{C}_{all}^*$ to denote the set of all classifiers such that $P(C(x) = +) = P(x(Class) = +)$. Suppose that we have a labeled dataset D and the perfect classifier P for this dataset; that is, $P(x) = x(Class)$ for all $x \in D$. P is an optimal classifier w.r.t. accuracy. Let D_0 and D_1 be defined as follows:

$$\begin{aligned} D_0 &:= \{x \in D \mid x(B) = b\} \\ D_1 &:= \{x \in D \mid x(B) \neq b\} \end{aligned}$$

and let d_0 and d_1 be respectively $|D_0|/|D|$ and $|D_1|/|D|$. The following theorem gives us some insight in the trade-off between accuracy and dependency in perfect classifiers, namely those that are DA-optimal in the set of *all* classifiers, and those that are DA-optimal in the set of all classifiers that does not change the class distribution:

Theorem 1. A classifier C is DA-optimal in $\mathcal{C}_{all}$ iff

$$acc(C) = 1 - \min(d_0, d_1)(dep(P) - dep(C))$$

A classifier C is DA-optimal in $\mathcal{C}_{all}^*$ iff

$$acc(C) = 1 - 2d_0d_1(dep(P) - dep(C))$$

As was claimed before, there is a trade-off between the accuracy of the DA-optimal classifiers and their dependency. This trade-off is linear; lowering the dependence level by 1% results in an accuracy decrease of $\min(d_0, d_1)\%$ and an accuracy decrease of $2d_0d_1\%$ if the class distribution needs to be maintained. These DA-optimal classifiers can be constructed from the perfect classifier.

The proof sketch for Theorem 1 can be found in Appendix A.

Imperfect Classifiers. Of course in reality we never have a perfect classifier to start from. From any given classifier C , however, we can easily construct another (non-deterministic) classifier by changing the predicted labels. For a given tuple x , the classifier $C[p_{0+}, p_{0-}, p_{1+}, p_{1-}]$ will assign the label $C(x)$ with probability $p_{x(B)x(Class)}$ and the other label with probability $1 - p_{x(B)x(Class)}$. That is, p_{bc}

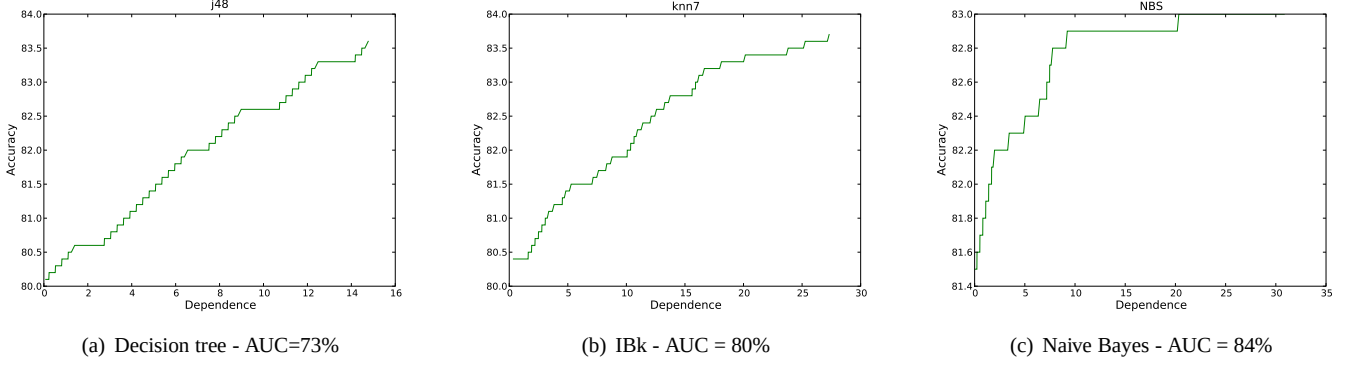


Figure 1. Trade-off between accuracy and dependence for the class $\mathcal{C}_{R(c_0, c_1)}$.

represents the probability that the label assigned by C to a tuple with $B = b$ and $Class = c$ will remain the same in the new classifier. Notice that the accuracy and dependency of this random classifier in fact represents the expected accuracy and dependency of all deterministic classifiers with $p_{0+}, p_{0-}, p_{1+}, p_{1-}$ correspondence with C . We will denote the class of all classifiers that can be derived from C in this way by $\mathcal{C}_C$. $\mathcal{C}_C^*$ will denote all classifiers C' in $\mathcal{C}_C$ for which it holds that $P(C'(x) = +) = P(C(x) = +)$. The following theorem characterizes the DA-optimal classifiers of $\mathcal{C}_C$ and of $\mathcal{C}_C^*$.

Theorem 2. *The classifier C' is DA-optimal in $\mathcal{C}_C$ iff*

$$acc(C) - acc(C') = \alpha(dep(C) - dep(C'))$$

with

$$\alpha := \min \left(d_0 \frac{tp_0 - fp_0}{tp_0 + fp_0}, d_1 \frac{tn_1 - fn_1}{tn_1 + fn_1} \right)$$

The classifier C' is DA-optimal in $\mathcal{C}_C^$ iff*

$$acc(C) - acc(C') = \beta(dep(C) - dep(C'))$$

with

$$\beta := d_0 d_1 \left[\frac{tp_0 - fp_0}{tp_0 + fp_0} + \frac{tn_1 - fn_1}{tn_1 + fn_1} \right]$$

tp_i (tn_i, fp_i, fn_i), $i = 0, 1$ denotes the true positive (true negative, false positive, false negative) rate in D_i .

Again we see a linear trade-off. This linear trade-off could be interpreted as bad news: no matter what we do, we will always have to trade in accuracy proportional to the decrease in dependence we want to achieve. Especially when the classes are balanced this is a high price to pay.

The proof sketch for Theorem 2 can be found in Appendix A.

Classifiers based on rankers. On the bright side, however, most classification models actually provide a score or probability for each tuple for being in the positive class instead of only giving the class label. This score allows us for a more careful choice of for which tuples to change the predicted label: instead of using a uniform weight for all tuples with the same predicted class and B -value the score can be used as follows. We dynamically set different cut-off c_0 and c_1 for respectively tuples with $B = 0$ and $B = 1$; for a ranker R , the classifier $R(c_0, c_1)$ will predict $+$ for a tuple x if $x(B) = 0$ and $R(x) \geq c_0$, or $x(B) = 1$ and $R(x) \geq c_1$. Otherwise $-$ is predicted. The class of all classifiers $R(c_0, c_1)$ will be denoted $\mathcal{C}_R$. Intuitively one expects that slight changes to the dependency will only incur minimal changes to the accuracy, as the tuples that are being changed are the least certain ones and hence actually sometimes a change will result in a better accuracy. The decrease in accuracy will thus no longer be linear in the change in dependency, but its rate will increase as the change in dependency increases, until in the end it becomes linear again, because the tuples we change will become increasingly more certain leading to a case similar to that of the perfect classifier. A full analytical exposition of this case, however, is far beyond the scope of this paper. Instead we tested this trade-off empirically. The results of this study are shown in Figure 1. In this figure the DA-optimal classifiers in the class $\mathcal{C}_R$ are shown for the Census-Income dataset [1]. The three rankers are obtained from a Decision Tree (J48), a Nearest Neighbor model (NN), and a Naive Bayesian Classifier (NB).

Conclusion. In this section the accuracy-dependence trade-off is clearly illustrated. It is theoretically shown that if we rely on classifiers, and not on rankers, the best we can hope for is a linear trade-off between accuracy and dependence. For important classes of classifiers the DA-optimal classifiers were explicitly constructed. Notice, however,

that the theoretical solutions proposed in this section violate our assumption A2; the classifiers $C[p_{0+}, p_{0-}, p_{1+}, p_{1-}]$ and $R(c_0, c_1)$ heavily use the attribute B to make their predictions. Therefore these optimal solutions are not suitable for our purposes. In the next section two solutions will be proposed that do not make use of the attribute B at prediction time, but only in the learning phase. The theoretically optimal solutions proposed in this section can be seen as “top-lines” which we never will be able to beat.

3. Solutions

In this section we propose two solutions to learn a classifier with dependence constraint that does not use the attribute B to make its predictions. Both solutions are based on removing the dependency from the training dataset. On this cleaned dataset a classifier can be learned. Our hypothesis is that, since the classifier is trained on balanced data, its predictions will be (more) balanced as well. The empirical evaluation in Section 4 will confirm this statement. The first approach we will present, called *Massaging the data*, is based on changing the class labels in order to remove the dependency between $B = b$ and $Class$. A preliminary version of this approach was presented in [5]. The second approach is less intrusive as it does not change the class labels. Instead, the dataset is re-sampled in such a way that the dependency is removed. This approach will be called *Reweighting*. In the experimental section we will also consider a third, very straightforward option: instead of cleaning the training set, the attribute B and its most correlated attributes are removed from the dataset. It will be shown, however, that this method is almost always inferior to the two methods proposed in this section.

3.1. Massaging

In *Massaging*, we want to remove the dependency between B and the class attribute from the dataset. In order to do this, we will change the labels of some objects x with $x(B) = b$ from $-$ to $+$, and the same number of objects with $x(B) \neq b$ is changed from $+$ to $-$. From the proof of Theorem 1 we know that in this way we can reduce the dependence with the minimal number of changes to the dataset while keeping the overall positive class ratio constant. The set *pr* of objects x with $x(B) = b$ and $x(Class) = -$ will be called the *promotion candidates* and the set *dem* of objects x with $x(B) \neq b$ and $x(Class) = +$ will be called the *demotion candidates*.

We will not randomly pick promotion and demotion candidates to relabel. Instead a ranker will be used to select the best tuples as follows. On the training data, a ranker R for ranking the object according to their positive class probability is learned; i.e., the higher on the ranking and object

Sex	Ethnicity	Highest Degree	Job Type	Cl.	Prob
m	native	h. school	board	+	98%
m	native	univ.	board	+	89%
m	native	h. school	board	+	98%
m	non-nat.	h. school	healthcare	+	69%
m	non-nat.	univ.	healthcare	-	30%
f	non-nat.	univ.	education	-	2%
f	native	h. school	education	-	40%
f	native	none	healthcare	+	76%
f	non-nat.	univ.	education	-	2%
f	native	h. school	board	+	93%

Table 2. Sample job-application relation with positive class probability.

x is, the more probable it is that $x(Class) = +$. With this ranker, the promotion candidates are sorted according to descending rank by R and the demotion candidates according to ascending rank. When selecting promotion and demotion candidates, first the top elements will be chosen. In this way, the objects closest to the decision border are selected first to be relabeled, leading to a minimal effect on the accuracy.

The modification of the training data is continued until the dependency in it becomes zero. The number M of modifications required to make the data dependency-free can be calculated by using the following formula:

$$M = \frac{(b \times \bar{b} \wedge +) - (\bar{b} \times b \wedge +)}{b + \bar{b}}$$

where b and $\bar{b}$ represent respectively the number of objects with $B = b$ and $B \neq b$ while $b \wedge +$ and $\bar{b} \wedge +$ are the number of objects with label $+$ and $B = b$ or $B \neq b$ respectively.

Example 1. Consider again the example Table 1. We want to learn a classifier to predict the class of objects for which the predictions are independent of $Sex = f$. In this example we rank the objects by their positive class probability given by a Naive Bayesian classification model. In Table 2, for reference, the positive class probabilities as given by this ranker are added to the table (calculated by using NBS implementation of Weka).

In the second step, we arrange the data separately for female applicant with class $-$ in descending order and for male applicants with class $+$ in ascending order with respect to their positive class probability. The ordered promotion and demotion candidates are given in Table 3.

The number M of labels of promotion and demotion can-

Sex	Ethnicity	Highest Degree	Job Type	Cl.	Prob
f	native	h. school	education	-	40%
f	non-nat.	univ.	education	-	2%
f	non-nat.	univ.	education	-	2%
Sex	Ethnicity	Highest Degree	Job Type	Cl.	Prob
m	non-nat.	h. school	healthcare	+	69%
m	native	univ.	board	+	89%
m	native	h. school	board	+	98%
m	native	h. school	board	+	98%

Table 3. Promotion candidates (negative objects with $Sex = f$ in descending order) and demotion candidates (positive objects with $Sex = m$ in ascending order)

didates we need to change equals:

$$\begin{aligned}
M &= \frac{(f \times (m \wedge +)) - (m \times (f \wedge +))}{f + m} \\
&= \frac{(5 \times 4) - (5 \times 2)}{5 + 5} = 1
\end{aligned}$$

So, 1 change from the promotion candidates list and one from the demotion candidates list will be required to make the data independent. We change the labels of the top promotion and demotion candidates (rows highlighted with the bold font in Table 2). After the labels for these instances are changed, the dependency level will decrease from 40% to 0%. So, the dataset (which will be used for future classifier learning) becomes dependency-free. $\square$

Algorithm. The pseudocode of our algorithm are given in Algorithm 1 and 2. Algorithm 1 describes the approach of changing the class labels and classifier learning, and Algorithm 2 describes the process of ordering the data objects separately for the promotion and demotion lists.

3.2. Reweighing

The *Massaging* approach is rather intrusive as it changes the labels of the objects. Our second approach does not have this disadvantage. Instead of relabeling the objects, different weights will be attached to them. For example, objects with $B = b$ and $Class = +$ will get higher weights than objects with $B = b$ and $Class = -$ and objects with $B \neq b$ and $Class = +$ will get lower weights than objects with $B \neq b$ and $Class = -$. According to these weights the objects will be sampled (with replacement) leading to a dataset without dependence. We will refer to this method as *Reweighing*. Again we will assume that we want to reduce

Algorithm 1: Changing the Class Labels

```

1 Input ( $D, B, b, +$ )
2 Output Classifier learnt on  $D$  without dependency

1:  $(pr, dem) := Rank(D, B, b, +)$ 
2:  $existDep := Dep_{B=b}(D)$ 
3: Calculate  $M$ , the number of necessary modifications based on  $existDep$ 
4: for  $M$  times do
5:   Select the data object from the top of  $pr$ 
6:   Change the class label of the selected object in  $D$ 
7:   Select the data object from the top of  $dem$ 
8:   Change the class label of the selected object in  $D$ 
9:   Remove the top elements of both  $pr$  and  $dem$ 
10: end for
11: Train a classifier on the modified  $D$ 
12: return Classifier with Independency Constraint

```

Algorithm 2: Rank

```

1 Input ( $D, B, b, +$ )
2 Output ( $pr, dem$ ): Two ordered lists of data objects on the basis of target class probability.

1: Learn a ranker  $R$  based on  $D$ 
2: Calculate the class probabilities  $R(x)$  for all  $x \in D$ 
3: Add all  $x$  in  $D$  with  $x(B) = b$  and  $x(c) \neq +$  into the list  $pr$  in descending order w.r.t.  $R(x)$ 
4: Add all  $x$  in  $D$  with  $x(B) \neq b$  and  $x(c) = +$  into the list  $dem$  in ascending order w.r.t.  $R(x)$ 
5: return ( $pr, dem$ )

```

the dependency to 0 while maintaining the overall positive class probability.

We discuss the idea of weight calculation by recalling some basic notions of probability theory with respect to this particular problem setting: If the dataset D is unbiased, in the sense that B and $Class$ are independent of each other, the expected probability $P_{exp}(b \wedge +)$ would be:

$$P_{exp}(b \wedge +) := b \times +$$

where b is the fraction of objects having $B = b$ and $+$ the fraction of tuples having $Class = +$. In reality, however, the actual probability

$$P_{act}(b \wedge +) := b \wedge +$$

might be different.

If the expected probability is higher than the actual probability value, it shows the bias towards class $-$ for $B = b$. We will assign weights to b with respect to class $+$. The

weight will be

$$W(B = b \mid x(Class) = +) := \frac{P_{exp}(b \wedge +)}{P_{act}(b \wedge +)}.$$

This weight of b for class $+$ will over-sample objects with $B = b$ for the class $+$. The weight of b for class $-$ will be

$$W(B = b \mid x(Class) = -) := \frac{P_{exp}(b \wedge -)}{P_{act}(b \wedge -)}$$

and the weights of $\bar{b}$ for class $+$ and $-$ will be

$$W(B = \bar{b} \mid x(Class) = +) := \frac{P_{exp}(\bar{b} \wedge +)}{P_{act}(\bar{b} \wedge +)}$$

$$W(B = \bar{b} \mid x(Class) = -) := \frac{P_{exp}(\bar{b} \wedge -)}{P_{act}(\bar{b} \wedge -)}.$$

In this way we assign to every tuple a weight according to its B - and $Class$ -values. The balanced dataset is then created by sampling the original training data, with replacement, according to the assigned weights. On this balanced dataset the dependency-free classifier is learned.

Example 2. Now we use *Reweighting* scheme to remove the dependency between Sex attribute and $Class$ attribute from the data of Table 1. We calculate a weight for each data object according to its B - and $Class$ -value. We observe that in this particular example where both $B = Sex$ and $Class$ attribute are binary attributes. Only four combinations between the values of B and $Class$ attribute are possible, i.e., $B = f$ or $B = m$ can have $Class$ -values $+$ or $-$. So weights of these four combinations will be sufficient for the whole data. For instance, we calculate the weight of a data object with $B = f$ and $Class = +$. We know that 50% objects have $B = f$ and 60% objects have $Class$ -value $+$, so the expected probability of the object should be:

$$P_{exp}(Sex = f \mid x(Class) = +) = 0.5 \times 0.6$$

but its actual probability is 20%. So the weight W will be:

$$\begin{aligned} W(Sex = f \mid x(Class) = +) &:= \frac{P_{exp}(Sex = f \wedge +)}{P_{act}(Sex = f \wedge +)} \\ &= \frac{0.5 \times 0.6}{0.2} = 1.5. \end{aligned}$$

Similarly weights of other combinations will be follow:

$$W(Sex = f \mid x(Class) = -) = 0.67$$

$$W(Sex = m \mid x(Class) = +) = 0.75$$

$$W(Sex = m \mid x(Class) = -) = 2.$$

The weight of each data object of the Table 1 is given in Table 4

Sex	Ethnicity	Highest Degree	Job Type	Cl.	Weight
m	native	h. school	board	+	0.75
m	native	univ.	board	+	0.75
m	native	h. school	board	+	0.75
m	non-nat.	h. school	healthcare	+	0.75
m	non-nat.	univ.	healthcare	-	2
f	non-nat.	univ.	education	-	0.67
f	native	h. school	education	-	0.67
f	native	none	healthcare	+	1.5
f	non-nat.	univ.	education	-	0.67
f	native	h. school	board	+	1.5

Table 4. Sample job-application relation with weights.

Algorithm 3: Reweighting

- 1 **Input** ($D, B, Class$)
 - 2 **Output** Classifier learnt on D without dependency
 - 1: $wtlist := Weights(D, B, Class)$
 - 2: **for** All data objects in D **do**
 - 3: Select the value v of B and c of $Class$ for the current data object
 - 4: Get the weight W from $wtlist$ for this combination of v and c and assign W to the data object.
 - 5: **end for**
 - 6: Train a classifier on the balanced D
 - 7: **return** Classifier with Independency Constraint
-

Algorithm. The pseudocode of the algorithm describing our *Reweighting* approach is given in Algorithm 3 and Algorithm 4 describes the procedure of weight calculation for each combination between B - and $Class$ -values.

4. Experiments

Claims. In this section we present experiments that support the following claims:

1. Due to the red-lining effect it is not enough to just remove the attribute B from the dataset in order to remove the dependence with the class attribute. Also removing B and the attributes that correlate with it does not have the desired effect, as either too much dependence remains or the accuracy is lowered too much.
2. Both proposed solutions get better results in the sense that they more optimally trade accuracy for independence. Especially the *Massaging* approach, if initiated with the right choice of ranker and base learner shows potential.

Algorithm 4: Weights

```
1 Input ( $D, B, Class$ )
2 Output The list of weights for each combination of
    $B$ - and  $Class$ -values.
3: for All  $v_i \in B$  do
4:   for All  $c_j \in C$  do
5:     Calculate expected probability
        $P_{exp}(v_i \wedge c_j) = P(v_i) \times P(c_j)$  in  $D$ 
6:     Calculate actual probability
        $P_{act}(v_i \wedge c_j) = P(v_i \wedge c_j)$  in  $D$ 
7:     Weight  $W$  for the data object with  $B = v_i$  and
        $Class = c_j$  will be  $\frac{P_{exp}(v_i \wedge c_j)}{P_{act}(v_i \wedge c_j)}$ 
8:     Add  $W$ ,  $v_i$  and  $c_j$  in  $wtlist$ 
9:   end for
10: end for
11: return  $wtlist$ 
```

3. More concretely, when the goal is to reduce the dependence to zero while maintaining a high accuracy, a good ranker with a base learner that is sensitive to small changes in the dataset seems to be the best choice.

Experimental setup. In our experiments we used the Census Income dataset and the German Credit datasets which are both available in the UCI ML-repository [1]. As the results and conclusions we can draw from them are very similar for both datasets, we only present the figures for the Census Income dataset. Census Income has 48842 instances of which we used only a random sample of 1/3 for reasons of efficiency. Census Income contains demographic information about people and the associated prediction task is to determine whether a person makes over 50K per year or not, i.e., income class *High* or *Low* will be predicted. We will denote income class *High* as $+$ and income class *Low* as $-$. Each data object is described by 14 attributes which include 8 categorical and 6 numerical attributes. We excluded the attribute *fnlwgt* from our experiments (as suggested in the documentation of the dataset). The other attributes in the dataset include: age, type of work, education, years of education, marital status, occupation, type of relationship (husband, wife, not in family), sex, race, native country, capital gain, capital loss and weekly working hours. We use $Sex = f$ as dependent attribute. In our sample of the dataset, 5421 citizens have $Sex = f$ and 10860 have $Sex = m$. This dependency between $Sex = f$ and $Class$ is as high as 19.13%; i.e.,

$$\begin{aligned} P(x(Class) = + | x(Sex) = m) \\ - P(x(Class) = + | x(Sex) = f) = 19.13\% \end{aligned}$$

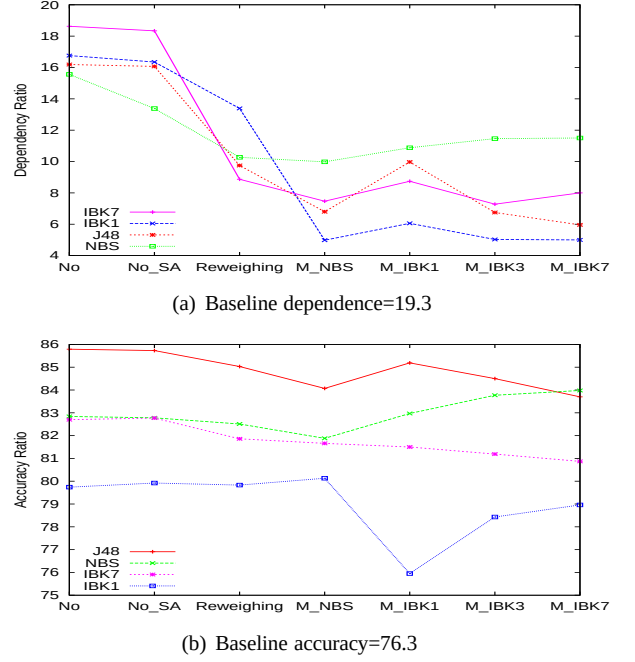


Figure 2. The results of 10-fold CV.

The goal is now to learn a classifier that has minimal dependence between $Sex = f$ and its predictions while maintaining a high accuracy. All reported accuracy numbers in the paper were obtained using 10-fold cross-validation and reflect the true accuracy; that is, on unaltered (non-massaged) test data.

4.1. Testing the Proposed Solutions

We conducted experiments to test our proposed solutions. We compare three different types of algorithms:

1. Two **baseline** approaches: an out-of-the-box classifier was learned on at the one hand the original data (labeled “No” in the graphs to reflect no *Preprocessing* technique was applied) and on the other hand the original data but with the attribute *Sex* removed (labeled “No_SA (*Sex* Attribute)” in the graphs).
2. The **Massaging** approach with different combinations of base learner and ranker. We consider four different rankers: one based on a Naïve Bayesian classifier (M_NBS), and three based on nearest neighbors with respectively 1, 3 and 7 neighbors (M_IBk1, M_IBk3, and M_IBk7). For the base classifiers that are learned on the massaged data, a Naïve Bayes Classifier (NBS) was used, two nearest neighbor classifiers with respectively 1 and 7 neighbors (IBk1 and IBk7), and a decision tree learner: the Weka implementation of the C4.5

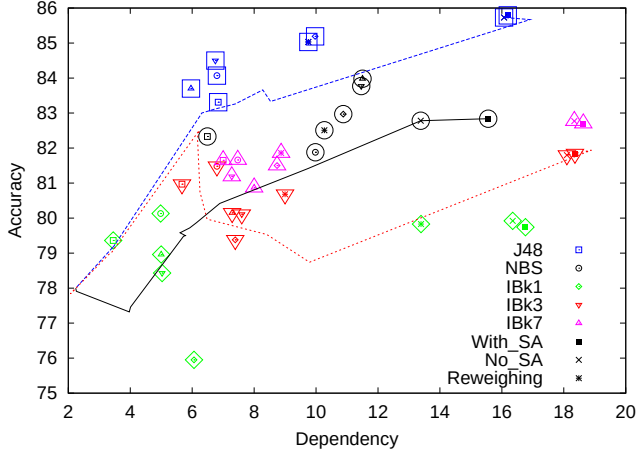


Figure 3. Accuracy-dependency trade-off comparison. Outer and inner symbol of each data point shows the corresponding base learner and preprocessing technique respectively.

classifier (J48). Many more combinations have been tested (including, e.g., also Adaboost and all possible combinations) but we will restrict to these choices as they present a good overview of the obtained results.

3. The **Reweighing** approach with different base classifiers (labeled “Reweighing” in the graphs).

In Figures 2(a) and 2(b), respectively the dependence and accuracy results for all algorithms under comparison are given. The X-axis shows the names of the data preprocessing techniques which have been applied to the training dataset to remove undesirable dependencies between *Sex* and *Income* class attribute. The resultant dependency has been given on the Y-axis of Figure 2(a) and the accuracy on the Y-axis of Figure 2(b). We observe that the classification models with independency constraint produce less dependent results as compared to the baseline algorithms; in Figure 2(a) we see that IBk1 classifies the future data objects with a dependence level of 16.76% which is lowered only slightly if the *Sex* attribute is removed. If *Massaging* is applied, however, the dependency level goes down to 4.98%. The dependency level always goes down when we apply our classifiers with independency constraint. Clearly, the choice of base learner and ranker (for *Massaging*) plays a very important role in dependency free classification. The accuracy drops to some extent because our test set contains these undesirable dependencies.

Figure 3 offers a good overview that allows us to quickly assess which of the combinations are DA-optimal (dependency-accuracy-optimal) in the class of the classifiers learned in our experiments. Each pictogram in this

figure represents a particular combination of a classification algorithm (shown by outer symbol) and corresponding preprocessing technique (shown by inner shape of the data point). For *Massaging*, the inner symbol will represent the corresponding ranker. On the X-axis we see the dependence and on the Y-axis, the accuracy. Thus, we can see the trade-off between accuracy and dependence for each combination. The closer we are to the top left corner the higher accuracy and the lower dependency we obtain. We observe that the top left area in the figure is occupied with the data points corresponding to the performance of *Massaging* approach. *Reweighing* approach falls behind *Massaging* but also shows reasonable performance. From Figure 3 we can see that our both approaches compare favorably to the baseline and the simplistic solutions: the three lines in the figure represent three classifiers (J48, NBS and IBk3 from the top to bottom) learned on the original dataset (the most top-right point in each line, denoted with *With_SA* symbol), the original dataset with the *Sex* attribute removed (denoted with *No_SA* symbol), the original dataset with the *Sex* attribute and the one (two, three, and so on) most correlated attribute(s) removed (that typically correspond to the further decrease in both accuracy and dependency). We can clearly see that this simplistic solution is always dominated by our classification with independency constraints approaches.

Overall, we hypothesize that *Massaging* has a more impact on an unstable classifier, e.g., J48 than stable classifiers, e.g., NBS. When we remove dependencies from training data, this effect is clearly transferred to future classification in case of unstable classifiers and both the dependency level and the accuracy goes down more than for a stable (noise-resistant) classifier. So, if the minimal dependence is the first priority, an unstable classifier is the better option and if the high accuracy is the main concern, a stable classifier might be more suitable. Of course the experiments conducted here are insufficient to draw this conclusion beyond doubt. To substantiate our hypothesis further, we conducted additional experiments where we used a k-nearest neighbor classifier. This classifier has the clear advantage that we can influence its stability with the parameter k : the higher k , the more stable it becomes. Figure 4 represent the results of the experiments with IBk as base learner and NBS as ranker for the *Massaging* approach. We changed the value of k for IBk from 1 to 19 to change its stability as a base classifier. We observe that the resultant dependency and accuracy increase with an increase of k which supports our claim.

5. Related Work

There are many relations with existing techniques from classification. Due to space restrictions, we only discuss the most relevant links. Despite the abundance of related works, none of them satisfactory solves the classification

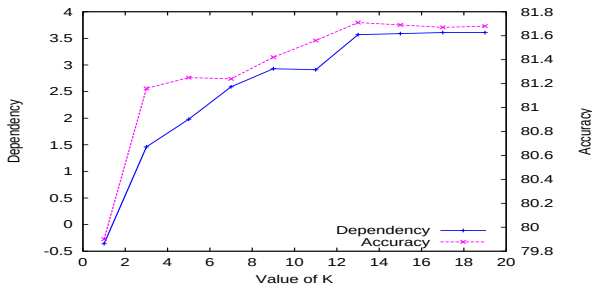


Figure 4. Accuracy and dependency comparison with NBS as a ranker and IBk as a base learner with different value of k .

with independence constraints problem.

In Constraint-Based Classification, next to a training dataset also some constraints on the model have been given. Only those models that satisfy the constraints are considered in model selection. For example, when learning a decision tree, an upper bound on the number of nodes in the tree can be imposed. Our proposed classification problem with independence constraints clearly fits into this framework. Most existing works on constraint based classification, however, impose purely syntactic constraints limiting, e.g., model complexity, or explicitly enforcing the predicted class for certain examples. One noteworthy exception is monotonic classification [3, 6], where the aim is to find a classification that is monotone in a given attribute.

In Cost-Sensitive and Utility-Bases learning [2, 4, 7], it is assumed that not all types of prediction errors are equal and not all examples are as important. The type of error (false positive versus false negative) determines the cost. Sometimes costs can also depend on individual examples. In cost-sensitive learning the goal is no longer to optimize the accuracy of the prediction, but rather the total cost. Our *Reweighting* technique can be seen as an instance of cost-sensitive learning in which, e.g., an object of class $+$ with $B = b$ gets a higher weight and hence an error for this object becomes more expensive.

In Discrimination-Aware Data Mining [9, 8] similar concepts of dependency are introduced and also the assumption of undesirable dependencies between the class and some sensitive attributes is made. These works, however, concentrate on identifying the discriminatory rules that are present in a dataset rather than on learning a classifier that avoids this dependency in future predictions. As such, discrimination-aware data mining and classification with independence constraints handle different problems.

6. Conclusion and Discussion

In this paper we presented the classification with independence constraints problem. Two approaches towards the

problem were proposed: *Massaging* and *Reweighting* the dataset. Both approaches remove the dependence from the training data and the claim is that a classifier learned on this unbiased data will be unbiased itself. Experimental evaluation shows that indeed this approach allows for removing dependency from the dataset more efficiently than simple methods such as, e.g., removing the dependent attribute from the training data. All methods have in common that to some extent accuracy must be traded-off for lowering the dependence. This trade-off was studied and confirmed theoretically.

As future work we identify two important main directions. In the first place we want to develop other methods, not based on changing the dataset, but rather on model selection to learn unbiased classifiers. A first step will be the development of dependency-aware decision trees. Secondly, we are also interested in extending the dependency model itself; in many cases, independence constraints as introduced above are too strong: often it is acceptable from an ethical and legal point of view to have a correlation between the sex of a person and the label given to him or her, as long as it can be explained by other attributes. Therefore it would be interesting refine our model to *Classification with Conditional Independence Constraints*.

To conclude, we believe that classification with independence constraints is a new and exciting area of research addressing a societally relevant problem.

References

- [1] A. Asuncion and D. Newman. UCI machine learning repository. 2007.
- [2] P. K. Chan and S. J. Stolfo. Toward scalable learning with non-uniform class and cost distributions: A case study in credit card fraud detection. In *Proc. ACM SIGKDD'98*, pages 164–168, 1998.
- [3] W. Duivesteijn and A. Feelders. Nearest neighbour classification with monotonicity constraints. In *Proc. ECML/PKDD'08*, pages 301–316. Springer, 2008.
- [4] C. Elkan. The foundations of cost-sensitive learning. In *Proc. IJCAI'01*, pages 973–978, 2001.
- [5] F. Kamiran and T. Calders. Classifying without discriminating. In *Proc. IC4'09*. IEEE press.
- [6] W. Kotlowski, K. Dembczynski, S. Greco, and R. Slowinski. Statistical model for rough set approach to multicriteria classification. In *Proc. ECML/PKDD'07*. Springer, 2007.
- [7] D. Margineantu and T. Dietterich. Learning decision trees for loss minimization in multi-class problems. Technical report, Dept. Comp. Science, Oregon State University, 1999.
- [8] D. Pedreschi, S. Ruggieri, and F. Turini. Measuring discrimination in socially-sensitive decision records. In *Proc. SIAM SDM'09*.
- [9] D. Pedreschi, S. Ruggieri, and F. Turini. Discrimination-aware data mining. In *Proc. ACM SIGKDD'08*, 2008.

A Appendix. Proof sketches

Proof sketch for Theorem 1. We denote the set of true negatives, true positives, false positives and false negatives of C by respectively TN , TP , FP , and FN . Their relative sizes will be denoted tn , tp , fp , fn respectively. That is: $tn = \frac{|TN|}{|D|}$. We also consider the split of D into $D_0 = \{t \in D \mid t.D = 0\}$ and $D_1 = \{t \in D \mid t.D = 1\}$, and denote the relative fractions $d_0 = \frac{|D_0|}{|D|}$ and $d_1 = \frac{|D_1|}{|D|}$. Similarly, TN_0 will denote $D_0 \cap TN$, and $tn_0 = \frac{|TN_0|}{|D_0|}$. With these conventions, we can express the accuracy and dependence of C as follows:

$$\begin{aligned} acc(C) &= tp + tn \\ &= d_0(tp_0 + tn_0) + d_1(tp_1 + tn_1) \\ dep(C) &= (tp_1 + fp_1) - (tp_0 + fp_0) \end{aligned}$$

A careful analysis of these formulas reveals that any DA-optimal classifier must have fn_0 and fp_1 equal to 0. Furthermore, accuracy can be lowered while keeping $dep(C)$ constant by decreasing tn_0 by ϵ and increasing tp_1 at the same time by the same amount ϵ . The effect of this is that $acc(C)$ increases or decreases depending on the relative magnitudes of d_0 and d_1 . Depending on which choice increases $acc(c)$, either tn_0 is maximized or minimized leading to the given bounds. $\square$

Proof sketch for Theorem 2. For the classifier C' , the true positive rate for D_0 , tp'_0 , will be:

$$tp'_0 = p_{0+}tp_0 + (1 - p_{0-})fn_0 ,$$

as there are two types of true positive predictions: on the one hand true positive predictions of C and that were not changed in C' (probability p_{0+}) and on the other hand false negative predictions of C that were changed in C' (probability $1 - p_{0-}$). For the other quantities similar identities exist. Using the same naming conventions as in the proof of Theorem 1, we now get:

$$\begin{aligned} acc(C') &= d_0(tp'_0 + tn'_0) + d_1(tp'_1 + tn'_1) \\ &= d_0(p_{0+}tp_0 + (1 - p_{0-})fn_0 + p_{0-}tn_0 + (1 - p_{0+})fp_0) \\ &\quad + d_1(p_{1+}tp_1 + (1 - p_{1-})fn_1 + p_{1-}tn_1 + (1 - p_{1+})fp_1) \\ dep(C') &= (tp'_1 + fp'_1) - (tp'_0 + fp'_0) \\ &= (p_{1+}tp_1 + (1 - p_{1-})fn_1 + p_{1+}fp_1 + (1 - p_{1-})tn_1) \\ &\quad - (p_{0+}tp_0 + (1 - p_{0-})fn_0 + p_{0+}fp_0 + (1 - p_{0-})tn_0) \end{aligned}$$

Similarly as in the proof of Theorem 1, we can show that for a DA-optimal classifier, $p_{0+} = p_{1-} = 1$; i.e., we never

change a positive prediction in D_0 to a negative one or a negative prediction in D_1 into a positive one. Depending on the exact true and false positive and negative rates and the sizes d_0 and d_1 , the optimal solutions are as given in the theorem. $\square$